

Q: write a C program to simulate the following CPU scheduling algorithm to turnaround time and waiting time. (Any one)

- (a) FCFS
- (b) STF
- (c) Priority
- (d) Round Robin (Experiment with different quantum sizes for RR algorithm)

A.

FCFS

```
#include <stdio.h>

int main()
{
    int n, i;
    int at[20], bt[20], ct[20], tat[20], wt[20];
    float total_wt = 0, total_tat = 0;
    printf("Enter no. of processes: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++)
    {
        printf("In Process P-1.d\n", i+1);
        printf("Arrival time: ");
        scanf("%d", &at[i]);
        printf("Burst time: ");
        scanf("%d", &bt[i]);
    }
    ct[0] = at[0] + bt[0];
    for (i = 1; i < n; i++)
    {
        if (ct[i-1] < at[i])
            ct[i] = at[i] + bt[i];
        else
            ct[i] = ct[i-1] + bt[i];
    }
```

```
for (i=0; i < n; i++) {
```

```
    tat[i] = ct[i] - at[i];
```

```
    wt = tat[i] - bt[i];
```

```
    total_tat += tat[i];
```

```
    total_wt += wt[i];
```

```
}
```

```
printf("Input AT\t BT\t CT\t TAT\t WT\n");
```

```
for (i=0; i < n; i++) {
```

```
    printf("P\t\t AT\t BT\t CT\t TAT\t WT\n");
```

```
    i+1, at[i], bt[i], ct[i], tat[i], wt[i]
```

```
}
```

```
printf("Average TAT = %.2f", total_tat / n);
```

```
printf("Average WT = %.2f", total_wt / n);
```

```
return;
```

```
}
```

output

Enter number of processes: 2

Process P1

Arrival time: 0

Burst Time: 6

Process P2

Arrival time: 1

Burst Time: 7

Process P3

Arrival Time: 2

Burst Time: 3

P	AT	BT	CT	TAT	WT
P1	0	6	6	6	0
P2	2	7	10	12	5
P3	2	3	16	14	11

Average TAT = $\frac{8+10+10}{3} = 10.67$

Average WT = $\frac{0+5+11}{3} = 5.33$

SF non preemptive & preemptive

```
#include <stdio.h>
```

```
int main()
```

```
{ int n, i, j, choice;
```

```
int at[20], bt[20], ct[20], tat[20], wt[20];
```

```
int rt[20], p[20];
```

```
float total_wt=0, total_tat=0;
```

```
printf("Enter no. of processes: ");
```

```
scanf("%d", &n);
```

```
for(i=0; i<n; i++){
```

```
printf("Process P%d\n", i+1);
```

```
printf("Arrival time: ");
```

```
scanf("%d", &at[i]);
```

```
printf("Burst time: ");
```

```
scanf("%d", &bt[i]);
```

```
p[i] = i+1;
```

```
rt[i] = bt[i];
```

```
} printf("\n 1. SF Non-preemptive");
```

```
printf("\n 2. SF Preemptive (SRIF)");
```

```
printf("\n Enter your choice: ");
```

```
scanf("%d", &choice);
```

```
if(choice == 1){
```

```
{ int temp;
```

```
for
```

```
for (i = 0; i < n - 1; i++) {
```

```
    for (j = i + 1; j < n; j++) {
```

```
        if (bt[i] > bt[j]) {
```

```
            temp = bt[i];
```

```
            bt[i] = bt[j];
```

```
            bt[j] = temp;
```

```
            temp = at[i];
```

```
            at[i] = at[j];
```

```
            at[j] = temp;
```

```
            temp = p[i];
```

```
            p[i] = p[j];
```

```
            p[j] = temp;
```

```
        }
```

```
    }
```

```
}
```

```
ct[0] = at[0] + bt[0];
```

```
for (i = 1; i < n; i++) {
```

```
    if (ct[i-1] < at[i]) {
```

```
        ct[i] = at[i] + bt[i];
```

```
    else {
```

```
        ct[i] = ct[i-1] + bt[i];
```

```
    }
```

```
}
```

```
else if (choice == 2) { // Dream phase
```

```
    int time = 0, completed = 0;
```

```
    int min, shortest;
```

```
    while (completed < n) {
```

```
        min = 9999;
```

```
        shortest = -1;
```

```
        for (i = 0; i < n; i++) {
```

```
            if (at[i] < time || (at[i] == time && rt[i] < min)) {
```

```
                min = rt[i];
```

```
                min = rt[i];
```

```

shortest = 1;
{
    {
        if (shortest == -1)?
            time++;
        }
        else?
            rt(shortest)--;
            time++;
            if (rt(shortest) == 0)?
                completed++;
                ct(shortest) = time;
            }
        }
    }
}
else?
    printf("Invalid choice!\n");
    return 0;
}

for (i = 0; i < n; i++)?
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
    total_tat += tat[i];
    total_wt += wt[i]; // calculating TAT & WT
}
printf("Input AT BT ET CT TAT WT\n");
for (i = 0; i < n; i++)?
    printf("P1 Q1 T1 D1 T1 A1 E1 C1 T1 W1\n",
        i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
}
return 0;
}

```

output

Enter no. of processes : 3

Process P1

Arrival time : 0

Burst time : 6

Process P2

Arrival time : 1

Burst time : 7

Process P3

Arrival time : 2

Burst time : 3

1. SJF non preemphive

2. SJF Preemphive

Enter choice : 1

P	AT	BT	CT	TAT	WT
P1	0	6	6	6	0
P2	1	7	13	13	6
P3	2	3	18	18	10

Average TAT = 10.33

Average WT = 5.00

✓

For average TAT, WT STF non-preemptive is better.

Q. Write a C Program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.

Priority (preemptive & non preemptive)
non preemptive

A. #include <stdio.h>

int main() {

int n, i, j;

int at[20], bt[20], pr[20], ct[20], tat[20], wt[20];

p[20];

int temp;

float total_wt = 0, total_tat = 0;

printf("Enter no. of processes: ");

scanf("%d", &n);

for(i = 0; i < n; i++) {

printf("Process %d\n", i + 1);

printf("Arrival time: ");

scanf("%d", &at[i]);

printf("Burst time: ");

scanf("%d", &bt[i]);

printf("Priority (Lower number = Higher Priority): ");

scanf("%d", &pr[i]);

p[i] = i + 1;

}

```
for (i=0; i<n-1; i++) {
```

```
    for (j=i+1; j<n; j++) {
```

```
        if (pr[i] > pr[j]) {
```

```
            temp = pr[i];
```

```
            pr[i] = pr[j];
```

```
            pr[j] = temp;
```

```
            temp = bt[i];
```

```
            bt[i] = bt[j];
```

```
            bt[j] = temp;
```

```
            temp = pc[i];
```

```
            pc[i] = pc[j];
```

```
            pc[j] = temp;
```

```
        }
```

```
    }
```

```
}
```

```
ct[0] = at[0] + bt[0];
```

```
for (i=1; i<n; i++) {
```

```
    if (ct[i-1] < at[i]) {
```

```
        ct[i] = at[i] + bt[i];
```

```
    } else {
```

```
        ct[i] = ct[i-1] + bt[i];
```

```
    }
```

```
for (i=0; i<n; i++) {
```

```
    tat[i] = ct[i] - at[i];
```

```
    wt[i] = tat[i] - bt[i];
```

```
total_wt = wt[i];
```

```
total_tat = tat[i];
```

```
}
```

```
printf("Input Arrival Time PR, PCT, TAT, WT\n");
for (i = 0; i < n; i++) {
```

```
printf("P, AT, BT, PR, CT, TAT, WT\n");
scanf("%d %d %d %d %d %d %d", &P[i], &AT[i], &BT[i], &PR[i], &CT[i], &TAT[i], &WT[i]);
```

```
}
```

```
printf("In Average TAT = %.2f", total_tat/n);
```

```
printf("In Average WT = %.2f", total_wt/n);
```

```
return 0;
```

```
}
```

Output:

Enter no. of processes = 2

Process P1

Arrival time = 0

Burst time = 6

Priority (Lower no. = Higher Priority) = 3

Process P2

Arrival time = 2

Burst time = 8

Priority (Lower no. = Higher Priority) = 1

P	AT	BT	PR	CT	TAT	WT
P1	0	6	3	6	6	0
P2	2	8	1	10	18	10
P3	5	1	2	10	5	4

Avg TAT = 9.33

Avg WT = 4.67

Process P3

Arrival time: 5

Burst time: 1

Priority: 2

preemptive

#include <stdio.h>

int main()

{
int n;

int at[20], bt[20], pr[20];

int rt[20], ct[20], tat[20], wr[20];

int time = 0, completed = 0;

int highest, min;

float total_wt = 0, total_tat = 0;

printf("Enter no. of processes: ");

scanf("%d", &n);

for(i=0; i<n; i++)

{ printf("In Process %d\n", i+1);

printf("Arrival time: ");

scanf("%d", &at[i]);

printf("Burst time: ");

scanf("%d", &bt[i]);

(i) printf("Priority (lower number = Higher priority).");

(i) scanf("%d", &pr[i]);

rt[i] = bt[i];

g (i) w = + w - rt[i]

```

while (completed < n) {
    min = 9999;
    highest = -1;
    for (i = 0; i < n; i++) {
        if (at[i] < time + rt[i] & rt[i] > 0 & pr[i] <
            min) {
            min = pr[i];
            highest = i;
        }
    }
}

```

```

if (highest == -1) {
    time++;
}
else {
    rt[highest]--;
    time++;
    if (rt[highest] == 0) {
        completed++;
        at[highest] = time;
    }
}
}

```

```

for (i = 0; i < n; i++) {
    tat[i] = at[i] + (n - at[i]);
    wt[i] = tat[i] - bt[i];
}

```

```

total_tat += tat[i];
total_wt += wt[i];
}

```

```
printf("Input Arrival Time & Burst Time\n");
for (i = 0; i < n; i++) {
    printf("Process ID: %d, Arrival Time: %d, Burst Time: %d\n",
           i+1, at[i], bt[i], pr[i], c[i], r[i], wt[i]);
}
```

}

```
printf("Average TAT = %.2f", total_tat/n);
printf("Average WT = %.2f", total_wt/n);
```

```
return 0;
```

}

EnterOutput

```
Enter no. of processes: 3
```

Process P1

Arrival time: 0

Burst time: 6

Priority: 3

Process P2

Arrival time: 2

Burst time: 7

Priority: 1

Process P3

Arrival time: 5

Burst time: 1

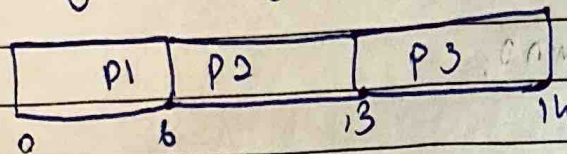
Priority: 2

P	AT	BT	PR	CT	TAT	WT
P ₁	0	6	3	14	14	8
P ₂	2	7	10	9	7	0
P ₃	5	1	2	10	5	4

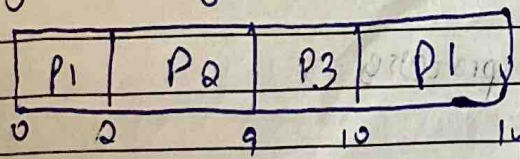
$$\text{Avg TAT} = 8.67$$

$$\text{Avg WT} = 4.00$$

Priority scheduling = non-preemptive - Gantt chart



Priority scheduling = preemptive - Gantt chart



decision, the average TAT & WT for the preemptive case is better. normally preemptive priority scheduling is preferred.

10/3/26

LAB-3

Q: write a C-Program to simulate the following CPU scheduling algo to find turn around time & waiting time for Round-Robin Algorithm

A: Code:

```
#include <stdio.h>

int main() {
    int n, tq;
    printf("Enter no. of processes: ");
    scanf("%d", &n);
    int at[n], bt[n], rem_bt[n], ct[n], tar[n], wt[n], rt[n];

    int first_response[n];

    for (int i = 0; i < n; i++) {
        printf("Enter AT & BT For P%d", i+1);
        scanf("%d %d", &at[i], &bt[i]);
        rem_bt[i] = bt[i];
        first_response[i] = -1;
    }

    printf("Enter Time quantum: ");
    scanf("%d", &tq);

    int time = 0, completed = 0;

    int gantt_p[100] = gantt_t[100];
    int idx = 0;

    while (completed < n) {
        int done = 1;
```

```
for (int i = 0; i < n; i++) {
    if (rem_bt[i] > 0 & ar[i] <= time) {
```

```
        done = 0;
```

```
        if (first_response[i] == -1) {
```

```
            first_response[i] = time;
```

```
            rt[i] = time - ar[i];
```

```
        }
```

```
        gantt_p[idx] = i;
```

```
        gantt_t[idx] = time;
```

```
        idx++;
```

```
        if (rem_bt[i] > tq) {
```

```
            time += tq;
```

```
            rem_bt[i] -= tq;
```

```
        } else {
```

```
            time += rem_bt[i];
```

```
            ct[i] = time;
```

```
            rem_bt[i] = 0;
```

```
            completed++;
```

```
    }
```

```
    if (done) {
```

```
        time++;
```

```
    }
```

```
    gantt_t[idx] = time;
```

```
    printf("\ngantt chart:\n");
```

```
    for (int i = 0; i < idx; i++) {
```

```
        printf("-----");
```

```
    }
```

```
    printf("\n");
```

```
    for (int i = 0; i < idx; i++) {
```

```
        printf("P-%d | " gantt_p[i] + 1);
```

```
    }
```

```

printf("%-10n");
for (int i=0; i<idx; i++) {
    printf("%-8d", gantt_t[i]);
}
printf("\n");
float total_wt = 0, total_tat = 0, total_rt = 0;
printf("In Process TAT BT CT RT\n");
for (int i=0; i<n; i++) {
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
    total_tat += tat[i];
    total_wt += wt[i];
    total_rt += rt[i];
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\t",
           i+1, at[i], bt[i], ct[i], tat[i], wt[i], rt[i],
           i+1, at[i], bt[i], ct[i], tat[i], wt[i], rt[i],
           i+1, at[i], bt[i], ct[i], tat[i], wt[i], rt[i]);
}
printf("In Average TAT = %.2f", total_tat/n);
printf("In Average WT = %.2f", total_wt/n);
printf("In Average RT = %.2f", total_rt/n);

return 0;
}

```

output :

Enter no. of processes : 3

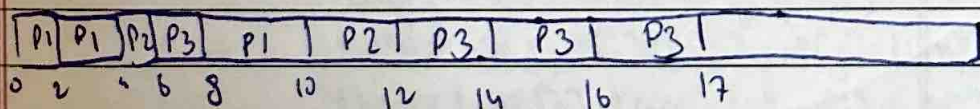
Enter AT & BT for P1: 0 6

Enter AT & BT for P2: 4 4

Enter AT & BT for P3: 6 7

Enter time quantum: 2

Gantt chart



Process	AT	BT	CT	TAT	WT	RT
P ₁	0	8	10	10	4	0
P ₂	4	4	12	8	4	0
P ₃	6	2	17	11	4	0

Avg TAT : 9.67

Avg WT : 4.00

Avg RT : 0.000

[Signature]
17/3/20

- Q: write a C program to simulate multilevel queue scheduling algorithm considering the following scenario. All the processes are divided into system processes and user processes. System processes are higher priority. use FCFs scheduling for the processes in each queue.

A: #include <stdio.h>

#define MAX 100

int main() {

int n;

printf("Enter number of processes ");

scanf("%d", &n);

int pid[MAX], at[MAX], bt[MAX], rt[MAX], type[MAX];

int cr[MAX], tar[MAX], wt[MAX];

for (int i = 0; i < n; i++) {

printf("Process %d\n", i);

pid[i] = i;

printf("Enter arrival time: ");

scanf("%d", &at[i]);

printf("Enter Burst time: ");

scanf("%d", &bt[i]);

printf("Enter type (0 = System, 1 = User): ");

scanf("%d", &type[i]);

rt[i] = bt[i];

}

for (int i = 0; i < n-1; i++) {

for (int j = i+1; j < n; j++) {

if (at[i] > at[j]) {

int temp;

temp = pid[i]; pid[i] = pid[j]; pid[j] = temp;

temp = at[i]; at[i] = at[j]; at[j] = temp;

temp = bt[i]; bt[i] = bt[j]; bt[j] = temp;

temp = rt[i]; rt[i] = rt[j]; rt[j] = temp;

temp = type[i]; type[i] = type[j]; type[j] = temp;

```

    }
}
int sysQ[Max], userQ[Max];

```

```

int sf = -1, sr = -1;

```

```

int uf = -1, ur = -1;

```

```

int time = 0, completed = 0, i = 0;

```

```

int current = -1;

```

```

while (completed < N) {

```

```

    while (i < N && arr[i] <= time) {

```

```

        if (type[i] == 0) {

```

```

            if (sf == -1) sf = 0;

```

```

            sysQ[sf++] = i;

```

```

        } else {

```

```

            if (uf == -1) uf = 0;

```

```

            userQ[uf++] = i;

```

```

        }

```

```

        i++;

```

```

    }

```

```

    if (current == -1) {

```

```

        if (sf != -1) {

```

```

            current = sysQ[sf];

```

```

            if (sf == sr) sf = sr = -1;

```

```

            else sf++;

```

```

        } else if (uf != -1) {

```

```

            current = userQ[uf];

```

```

            if (uf == ur) uf = ur = -1;

```

```

            else uf++;

```

```

        } else {

```

```

            time++;

```

```

            continue;

```

```

        }

```

```

    }

```

```

rt[current] = --;
time++;
if (rt[current] == 0) {
    ct[current] = time;
    completed++;
    current = --1;
}
}
float total_wt = 0, total_tat = 0;
for (int i = 0; i < n; i++) {
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
    total_wt += wt[i];
    total_tat += tat[i];
}
printf("1. dlt 1. slt 1. dlt 1. dlt 1. dlt 1. dlt 1. d\n");
for (int i = 0; i < n; i++) {
    printf("1. dlt 1. slt 1. dlt 1. dlt 1. dlt 1. dlt 1. d\n",
        pid[i],
        (types[i] == 0) ? "System" : "user",
        at[i], bt[i], ct[i], wt[i], tat[i]);
}
printf("In Average Waiting Time: 1.2f", total_wt/n);
printf("In Average Turnaround Time: 1.2f", total_tat/n);
return 0;

```

}

Enter number of Processes: 3

Process 0

Enter Arrival time: 0

Enter Burst time: 7

Enter type (0=System, 1=User): 1

Process 1

Enter Arrival time: 4

Enter Burst time: 2

Enter type (0=System, 1=User): 0

Process 2

Enter Arrival time: 3

Enter Burst time: 5

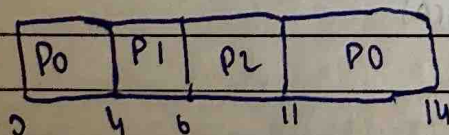
Enter type (0=System, 1=User): 1

ID	Type	AT	BT	CT	WT	TAT
0	User	0	7	14	7	14
2	User	3	5	11	3	8
1	System	4	2	6	0	2

Average Waiting time: 3.33

Average Turnaround Time: 8.00

Gantt chart



Q. Write a C program to simulate Real-Time CPU Scheduling algorithms:

(a) Rate - Monotonic

(b) Earliest - deadline first

A: #include <stdio.h>

int main()

{
int n;

printf("Enter number of processes: ");

scanf("%d", &n);

int burst[20], deadline[20], period[20];

int ct[20], wt[20], tat[20];

for (int i = 0; i < n; i++)

{
printf("Process %d: \n", i);

printf("Burst time: ");

scanf("%d", &burst[i]);

printf("Deadline (for EDF): ");

scanf("%d", &deadline[i]);

printf("Period (For RM): ");

scanf("%d", &period[i]);

ct[i] = 0;

}

printf("\n Earliest Deadline first (EDF) scheduling \n");

printf("ID BT Deadline CT WT TAT \n");

int time = 0;

for (int i = 0; i < n; i++)

{
int min_deadline = 9999, idx = -1;

for (int j = 0; j < n; j++)

{
if (deadline[j] < min_deadline && ct[j] == 0)

{
min_deadline = deadline[j];

idx = j;

}

}

```

time += burst[idx];
ct[idx] = time;
tar[idx] = ct[idx];
wt[idx] = tar[idx] - burst[idx];
printf("i.d i.d i.d i.d i.d i.d\n");
idx, burst[idx], deadline[idx], ct[idx],
wt[idx], tar[idx]);
}

```

printf("In Rate Monotonic Scheduling (RMS)\n");

printf("IO BT period CT WT TAT\n");

time = 0;

for (int i = 0; i < n; i++) {

int min_period = 9999; idx = -1;

for (int j = 0; j < n; j++) {

if (period[j] < min_period && ct[j] < min_period) {

min_period = period[j];

idx = j;

} }

time += burst[idx];

ct[idx] = time;

tar[idx] = ct[idx];

wt[idx] = tar[idx] - burst[idx];

printf("i.d i.d i.d i.d i.d i.d\n");

idx, burst[idx], period[idx], ct[idx],

wt[idx], tar[idx]);

ct[idx] += 2;

return 0;

}

Output

Enter number of processes: 3

Enter process details:

Process 0:

Burst time: 3

Deadline (For EDF): 10

Period (For RMS): 6

Process 1:

Burst time: 2

Deadline (For EDF): 5

Period (For RMS): 4

Process 2:

Burst time: 1

Deadline (For EDF): 5

Period (For RMS): 3

Earliest Deadline First (EDF) scheduling

ID	BT	Deadline	CT	WT	TAT
1	2	5	2	0	2
2	1	8	3	2	3
0	3	10	6	3	1

Rate Monotonic Scheduling (RMS)

ID	BT	Period	CT	WT	TAT
2	1	3	1	0	1
1	2	4	3	3	3
0	3	6	6	3	6

31/3/20

- Q. write a C program to simulate:
- Producer-consumer problem using semaphores
 - Dining-Philosopher's problem

```
A: #include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <stdlib.h>
#define N 5
#define BUFFER 3
```

```
int buffer[BUFFER];
int n = 0, out = 0;
```

```
sem_t empty, full, mutex;
```

```
void* producer(void * arg) {
    for (int i = 0; i < 5; i++) {
        sem_wait(&empty);
        sem_wait(&mutex);
        buffer[in] = i;
        printf("Produced: %d at buffer[%d]\n", i, in);
        in = (in + 1) % BUFFER;
        sem_post(&mutex);
        sem_post(&full);
        usleep(10000);
    }
    return NULL;
}
```

void * consumer (void * arg) {

int i = 0; int * p = (int *) arg;

while (1) {

sem_wait (& full);

sem_wait (& mutex);

int item = buffer[out];

printf ("consumed = %d from buffer (%d)\n", item, out);

out = (out + 1) % BUFFER;

sem_post (& mutex);

sem_post (& empty);

}

return NULL;

}

void run_p_c () {

pthread_t p, c;

sem_init (& empty, 0, BUFFER);

sem_init (& full, 0, 0);

sem_init (& mutex, 0, 1);

pthread_create (& p, NULL, producer, NULL);

pthread_create (& c, NULL, consumer, NULL);

pthread_join (p, NULL);

pthread_join (c, NULL);

sem_destroy (& empty);

sem_destroy (& full);

sem_destroy (& mutex);

}

sem_t co(N);

sem_t con;

void * philosopher (void * num) {

int id = (int *) num;

printf ("philosopher %d is thinking.\n", id);

```

sleep(1);
sem_wait(&room);
printf("Philosopher %d picked up left Fork %d\n", id);
sem_wait(&cs[id]);
printf("Philosopher %d picked up right fork %d\n", id,
      (id+1)%N);
sem_wait(&cs[(id+1)%N]);
printf("Philosopher %d is eating\n", id);
sleep(1);
printf("Philosopher %d put down forks %d & %d\n",
      id, id, (id+1)%N);
sem_post(&cs[id]);
sem_post(&cs[(id+1)%N]);
sem_post(&room);
return NULL;

```

}

```

void run_d_p() {
    pthread_t ph[N];
    int id[N];
    sem_init(&room, 0, N-1);
    for (int i=0; i<N; i++)
        sem_init(&cs[i], 0, 1);
    for (int i=0; i<N; i++) {
        id[i] = i;
        pthread_create(&ph[i], NULL, philosopher, &id[i]);
        //sleep
        //sleep(1500000);
    }
    for (int i=0; i<N; i++)
        pthread_join(ph[i], NULL);
    for (int i=0; i<N; i++)
        sem_destroy(&cs[i]);
    sem_destroy(&room);
}

```

}

```
int main() {
    int choice;
    while(1) {
        printf("1. PC In 2. OP In 3. Exit In ");
        printf("Enter choice: ");
        scanf("%d", &choice);
        switch(choice) {
            case 1: run_p_cc();
                    break;
            case 2: run_p_cc() run_d_pc();
                    break;
            case 3: exit(0);
            default: print("Invalid");
        }
    }
    return 0;
}
```

Tracing

Action	Buffer	mutex	empty	Full
P1	1 - -	1	2	1
C1	- - -	3	3	0
P2	- 2 -	1	2	1
C2	- - -	3	3	0
P3	- - 3	1	2	1
P4	4 - 3	1	1	2
C3	4 - -	1	2	1

21/4/26

Q. Write a C program to simulate Bankers Algorithm, Safety Algorithm.

```
A: #include <stdio.h>
#include <stdlib.h>

int main() {
    int n, m;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resources: ");
    scanf("%d", &m);

    int allocation[n][m], max[n][m], need[n][m];
    int available[m];

    printf("Enter allocation matrix: \n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            scanf("%d", &allocation[i][j]);
        }
    }

    printf("Enter maximum Demand matrix: \n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            scanf("%d", &max[i][j]);
        }
    }

    printf("Enter available resources: \n");
    for (int i = 0; i < m; i++) {
        scanf("%d", &available[i]);
    }
}
```

```

for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        need[i][j] = max(C[i][j]) - allocation[i][j];
    }
}

```

```

}
int work[m];

```

```

bool finish[n];

```

```

int safeSeq[n];

```

```

for (int i = 0; i < m; i++)

```

```

    work[i] = available[i];

```

```

    for (int j = 0; j < n; j++)

```

```

        finish[j] = false;

```

```

    int count = 0;

```

```

    while (count < n) {

```

```

        bool found = false;

```

```

        for (int i = 0; i < n; i++) {

```

```

            if (!finish[i]) {

```

```

                bool canExecute = true;

```

```

                for (int j = 0; j < m; j++) {

```

```

                    if (need[i][j] > work[j]) {

```

```

                        canExecute = false;

```

```

                        break;

```

```

                    }

```

```

                }

```

```

            if (canExecute) {

```

```

                for (int j = 0; j < m; j++)

```

```

                    work[j] += allocation[i][j];

```

```

                safeSeq[count++] = i;

```

```

                finish[i] = true;

```

```

                found = true;

```

```

            }

```

```

        }

```

```

    }

```

```
if (i < n) {
```

```
    printf("In System is 'n' in an unsafe  
    state (deadlock possible) 'n'");
```

```
    return 0;
```

```
}
```

```
{
```

```
    printf("In System is in a safe state 'n'");
```

```
    printf("Safe sequence is: ");
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf("%d ", safe_seq[i]);
```

```
        if (i != n-1)
```

```
            printf(" ");
```

```
}
```

```
    printf("In ");
```

```
    return 0;
```

```
}
```

Output:

no. of process: 5

no. of resource: 4

Enter allocation matrix

0 0 1 2

1 0 0 0

1 3 5 4

0 6 3 2

0 0 1 4

Enter maximum Demand matrix

0 0 1 2

1 7 5 0

2 3 5 6

0 6 5 2

0 6 5 6

Enter available resources

1 5 2 0

Safe state

sequence : $P_0 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4 \rightarrow P_1$

[Signature]
22/11/26

Q: Resource Request algorithm

A: #include <stdio.h>

#include <stdbool.h>

bool safetyAlgorithm(int n, int m, int allocation[n][m],
int need[n][m], int available[m]);

int work[m];

bool finish[n];

int safeseg[n];

for (int i = 0; i < m; i++) {

work[i] = available[i];

}

for (int i = 0; i < n; i++) {

finish[i] = false;

}

int count = 0;

while (count < n) {

bool found = false;

for (int i = 0; i < n; i++) {

if (!finish[i]) {

bool canExecute = true;

for (int j = 0; j < m; j++) {

if (need[i][j] > work[j]) {

canExecute = false;

break;

}

}

if (canExecute) {

for (int j = 0; j < m; j++) {

work[j] += allocation[i][j];

safeseg[count++] = i;

finish[i] = true;

found = true;

}

}

}

```

    if (!found) {
        return false;
    }
}

```

```

printf("\nSystem is in a safe state. In safe sequence is:");

```

```

for (int i = 0; i < n; i++) {

```

```

    printf("%d ", safeSeq[i]);

```

```

    if (i != n - 1) printf(" ");

```

```

}

```

```

printf("\n");

```

```

return true;

```

```

}

```

```

void resourceRequest(int n, int m, int allocation[n][m], int
    need[n][m], int available[m]) {

```

```

    int process;

```

```

    printf("\nEnter Process number (making request 0-%d):",
        n - 1);

```

```

    scanf("%d", &process);

```

```

    int request[m];

```

```

    printf("Enter request vector:\n");

```

```

    for (int i = 0; i < m; i++) {

```

```

        scanf("%d", &request[i]);

```

```

    }
    for (int i = 0; i < m; i++) {

```

```

        if (request[i] > need[process][i]) {

```

```

            printf("\nError: Process has exceeded its maximum
                claim.\n");

```

```

            return;

```

```

        }
    }

```

```

}

```

```

for (int i = 0; i < m; i++) {

```

```

    available[i] -= request[i];

```

```

    allocation[process][i] += request[i];

```

```

    need[process][i] -= request[i];
}

```

```

if (safetyAlgorithm(n, m, allocation, need, available))
    printf("\n Request can be granted safely.\n");
} else {

```

```

    printf("\n Request cannot be granted (unsafe)\n");

```

```

    for (int i = 0; i < m; i++) {

```

```

        available[i] += request[i];

```

```

        allocation[process][i] -= request[i];

```

```

        need[process][i] += request[i];
    }
}

```

```

int main() {

```

```

    int n, m;
    printf("Enter number of processes: ");

```

```

    scanf("%d", &n);

```

```

    printf("Enter number of resources: ");

```

```

    scanf("%d", &m);

```

```

    int allocation[m][m], max[m][m], need[m][m];

```

```

    int available[m];

```

```

    printf("Enter Allocation matrix:\n");

```

```

    for (int i = 0; i < n; i++) {

```

```

        for (int j = 0; j < m; j++) {

```

```

            scanf("%d", &allocation[i][j]);
        }

```

```

    printf("Enter maximum Demand matrix:\n");

```

```

    for (int i = 0; i < n; i++) {

```

```

        for (int j = 0; j < m; j++) {

```

```

            scanf("%d", &available[j]);
        }
    }
}

```

```

print C "Enter available resources:\n";
for (int i=0; i<m; i++)
    scanf ("%d", &available[i]);
for (int i=0; i<n; i++)
    for (int j=0; j<m; j++)
        need[i][j] = max[i][j] - allocation[i][j];

```

```

if (C: safetyAlgorithm(n, m, allocation, need, available))
    printf ("System is in an UNSAFE state initially\n");
    return 0;
}

```

```

(resourceRequest(n, m, allocation, need, available):
    return 0;
}

```

```

olp: Enter no. of processes: 5
Enter no. of resources: 3
Enter Allocation matrix:
0 0 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter request vector: 1 0 2
Request can be granted
Safe sequence: P1 -> P3 -> P4 -> P0
               -> P2

```

Enter max matrix:

7 5 3

(3) (2) (2)

1 0 2

2 2 2

4 3 3

Enter available resources:

3 3 2

System is in safe state

Safe sequence P: P1 -> P3 -> P4 -> P0 -> P2

5/5/26

a. Deadlock detection algorithm

```
A: #include <stdio.h>
```

```
#define MAX 10
```

```
int main()
```

```
{
    int n, m;
```

```
    int allocation[MAX][MAX], request[MAX][MAX], available;
```

```
    int work[MAX], finish[MAX];
```

```
    int i, j, k;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter number of resources types: ");
```

```
    scanf("%d", &m);
```

```
    printf("Enter allocation matrix:\n");
```

```
    for (i = 0; i < n; i++)
```

```
    {
        for (j = 0; j < m; j++)
```

```
        {
            scanf("%d", &allocation[i][j]);
```

```
        }
    }
```

```
    printf("Enter request matrix:\n");
```

```
    for (i = 0; i < n; i++)
```

```
    {
        for (j = 0; j < m; j++)
```

```
        {
            scanf("%d", &request[i][j]);
```

```
        }
    }
```

```
}
```

```
    printf("Enter available resources:\n");
```

```
    for (j = 0; j < m; j++)
```

```
    {
        scanf("%d", &available[j]);
```

```
    }
```

```

for (j=0; j<m; j++) {
    work[j] = available[j];
}

```

```

for (i=0; i<n; i++) {
    finish[i] = 0;
}

```

```

int found;

```

```

do {

```

```

    found = 0;

```

```

    for (i=0; i<n; i++) {

```

```

        if (finish[i] == 0) {

```

```

            int canExecute = 1;

```

```

            for (j=0; j<m; j++) {

```

```

                if (request[i][j] > work[j]) {

```

```

                    canExecute = 0;

```

```

                    break;

```

```

                }

```

```

            }

```

```

        } if (canExecute) {

```

```

            for (k=0; k<m; k++) {

```

```

                work[k] += allocation[i][k];

```

```

            }

```

```

            finish[i] = 1;

```

```

            found = 1;

```

```

        }

```

```

    }

```

```

}

```

```

while (bank);

```

```

int deadlock = 0;

```

```

printf("No deadlock of Process: ");

```

```

for (i=0; i<n; i++) {
    if (finish[i] == 0) {
        printf("p%d.d", i);
        deadlock = 1;
    }
}

```

```

if (deadlock == 0)
    printf("None (no Deadlock)\n");
else
    printf("System is in a Deadlock!\n");

```

```

return 0;
}

```

o/p: Enter no. of processes: 4

Enter no. of resources: 3

Enter Allocation matrix:

1 0 2

2 1 1

1 0 3

1 2 2

Enter request matrix:

0 0 0

0 0 2

0 0 0

3 3 0

Enter available resources:

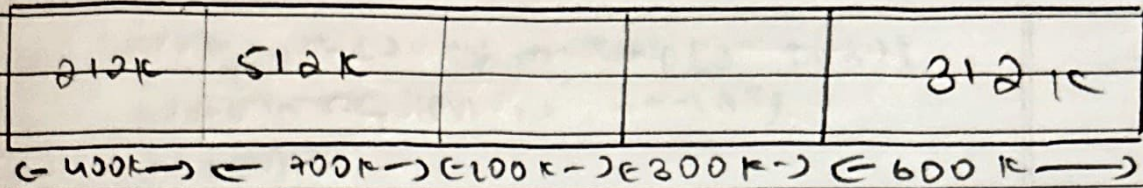
0 0 0

Deadlocked Process: P3

System is in Deadlock!

Q. Given a free memory partition 400K, 700K, 200K, 300K & 600K ^{in order}. How would each of first fit, best fit, & worst fit algorithms work to place process of 212K, 512K, 312K, 526K in order.

A: First Fit



526K cannot be allocated
with memory.

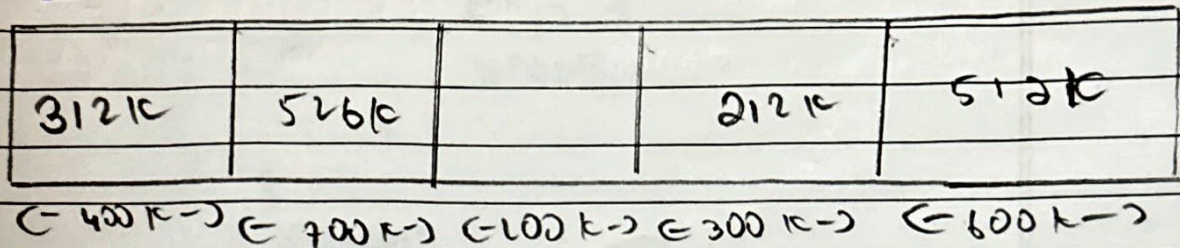
P₁ - 212K

P₂ - 512K

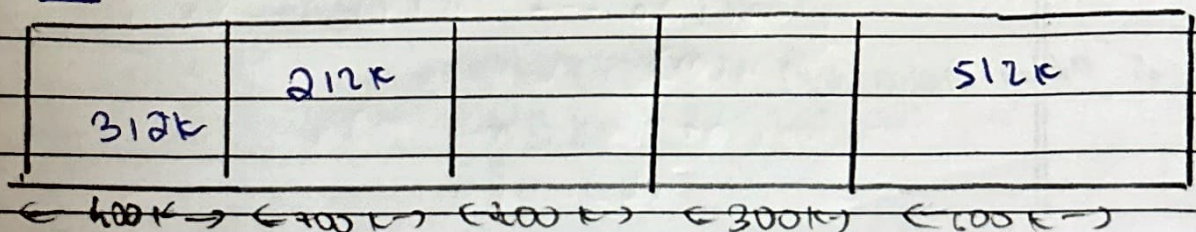
P₃ - 312K

P₄ - 526K

Best Fit



Worst Fit



526K cannot be allocated
with memory.

Q: Write a program to simulate the following contiguous memory allocation techniques

- (a) First Fit
- (b) Best Fit
- (c) Worst Fit

```

A. #include <stdio.h>

void FirstFit(int b[], int m, int p[], int n){
    printf("\n --- First Fit --- \n");
    for (int i = 0; i < m; i++){
        b[i] = b[i];
        for (int j = 0; j < n; j++){
            if (b[i] >= p[j]){
                b[i] = b[i] - p[j];
                break;
            }
        }
        if (j < n)
            printf("%d KB -> Block (%d KB) \n", p[j], b[i]);
        else
            printf("%d KB -> Not Allocated \n", p[j]);
    }
}
    
```

```

void bestFit (int b[], int m, int p[], int n) {
    printf("\n --- Best Fit --- \n");
    int t[m];
    for (int i=0; i<m; i++)
        t[i] = b[i];
    for (int i=0; i<n; i++) {
        int best = -1;
        for (int j=0; j<m; j++) {
            if (t[j] >= p[i])
                if (best == -1 || t[j] < t[best])
                    best = j;
        }
        if (best != -1) {
            printf("P%d (%dK) -> Block (%dK)\n", i+1, p[i], b[best]);
            t[best] = 0;
        } else {
            printf("P%d (%dK) -> Not Allocated\n", i+1, p[i]);
        }
    }
}

```

```

void worstFit (int b[], int m, int p[], int n) {
    printf("\n --- Worst Fit --- \n");
    int t[m];
    for (int i=0; i<m; i++)
        t[i] = b[i];
    for (int i=0; i<n; i++) {
        int worst = -1;
        for (int j=0; j<m; j++) {
            if (t[j] >= p[i])
                if (worst == -1 || t[j] > t[worst])
                    worst = j;
        }
        if (worst != -1) {
            printf("P%d (%dK) -> Block (%dK)\n", i+1, p[i], b[worst]);
            t[worst] = 0;
        } else {
            printf("P%d (%dK) -> Not Allocated\n", i+1, p[i]);
        }
    }
}

```

```

if (worst != -1) {
    printf("P%d (%dK) -> Block (%dK)\n", i+1, p[i], b[worst]);
    t[worst] = 0;
} else {
    printf("P%d (%dK) -> Not Allocated\n", i+1, p[i]);
}
}
}

```

Output:

```

--- First Fit ---
P1 (212K) -> Block
P2 (512K) -> Block
P3 (312K) -> Block
P4 (526K) -> Not

```

```

--- Best Fit ---
P1 (212K) -> Block
P2 (512K) -> Block
P3 (312K) -> Block
P4 (526K) -> Not

```

```

if (worst != -1) {
    printf("P%d (%dK) -> Block (%dK) \n",
           (i+1), p[i], b[worst]);
    b[worst] = 0;
} else {
    printf("P%d (%dK) -> Not allocated \n",
           i+1, p[i]);
}
}
}

```

```

int main() {
    int blocks[] = {400, 700, 200, 300, 600};
    int processes[] = {212, 512, 312, 516};
    int m=5, n=4;
    firstFit(blocks, m, processes, n);
    bestFit(blocks, m, processes, n);
    worstFit(blocks, m, processes, n);
    return 0;
}

```

Output:

--- First fit ---

P1 (212K) -> Block (400K)
 P2 (512K) -> Block (700K)
 P3 (312K) -> Block (600K)
 P4 (516K) -> Not allocated

--- Best fit ---

P1 (212K) -> Block (300K)
 P2 (512K) -> Block (600K)
 P3 (312K) -> Block (400K)
 P4 (516K) -> Block (700K)

[Signature]
12/5/20

--- worst fit ---

P1(212K) $\rightarrow$ Block (400K)

P2(512K) $\rightarrow$ Block (600K)

P3(312K) $\rightarrow$ Block (400K)

P4(526K) $\rightarrow$ Not allowed.

12/5/26

- Q. Write a C program to simulate page replacement algorithm,
 (a) FIFO
 (b) LRU
 (c) optimal.

```
#include <stdio.h>
#include <string.h>

#define Frames 3
#define pages 12

int pages[pages] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3};

int inFrames(int Frames[], int page) {
    for (int i = 0; i < Frames; i++)
        if (Frames[i] == page) return 1;
    return 0;
}

void FIFO() {
    int Frames[Frames] = { -1, -1, -1 };
    int faults = 0, ptr = 0;
    printf("\n --- FIFO --- \n");
    for (int i = 0; i < pages; i++) {
        if (!inFrames(Frames, pages[i])) {
            Frames[ptr] = pages[i];
            ptr = (ptr + 1) % Frames;
            faults++;
            printf("Page %d -> [ %d %d %d ] FAULT\n",
                pages[i], Frames[0], Frames[1], Frames[2]);
        } else {
            printf("Page %d -> [ %d %d %d ] HIT\n",
                pages[i], Frames[0], Frames[1], Frames[2]);
        }
    }
}
```

```

    pages[i], frames[0], frames[1], frames[2]
}
}
printf("Total Faults: %d\n", faults);
}

```

void LRU()

~~int~~ ~~fault~~

int frames[FRAMES] = {-1, -1, -1}

int lastused[FRAMES] = {0};

int faults = 0;

printf("\n--- LRU ---\n");

for (int i = 0; i < PAGES; i++) {

if (!inFrames(frames, pages[i])) {

int replace = 0;

for (int j = 0; j < FRAMES; j++) {

if (frames[j] == -1) {

replace = j;

break;

}

if (lastused[j] < lastused[replace])

replace = j;

}

frames[replace] = pages[i];

lastused[replace] = i;

faults++;

printf("Page %d is (%d %d %d) FAULT\n",

pages[i], frames[0], frames[1], frames[2])

else {

for (int j = 0; j < FRAMES; j++)

if (frames[j] == pages[i])

lastused[j] = i;

printf("Page %d is (%d %d %d) HIT\n",

```

    pages[i], frames[i], frames[i], frames[i]);
}

```

```

    printf("Total Faults = %d\n", faults);
}

```

```

void optimal() {
    int frames[FRAMES] = {-1, -1, -1};
    int faults = 0, filled = 0;
    printf("\n---OPTIMAL---\n");
    for (int i = 0; i < PAGES; i++) {
        if (filled < FRAMES) {
            frames[filled++] = pages[i];
        } else {
            int replace = 0, farthest = -1;
            for (int j = 0; j < FRAMES; j++) {
                int nextUse = PAGES;
                for (int k = i+1; k < PAGES; k++) {
                    if (pages[k] == frames[j]) {
                        nextUse = k; break;
                    }
                }
                if (nextUse < farthest) {
                    farthest = nextUse; replace = j;
                }
                frames[replace] = pages[i];
            }
            faults++;
            printf("Page %d -> [%d %d %d] FAULT\n",
                pages[i], frames[0], frames[1], frames[2]);
        }
        printf("Page %d -> [%d %d %d] HIT\n", pages[i],
            frames[0], frames[1], frames[2]);
    }
}

```

```
printf("Total faults : %d\n", faults);
```

```
}
```

```
int main () {
```

```
printf("Page string : 7 0 1 2 0 3 0 4 2 3 0 3");
```

```
printf("In frames : %d", FRAMES);
```

```
FIFO();
```

```
LRU();
```

```
optimal();
```

```
return 0;
```

```
}
```

output:

Page string : 7 0 1 2 0 3 0 4 2 3 0 3

frames: 3

FIFO

7 → [7] -> Fault

0 → [7 0] -> Fault

1 → [7 0 1] -> Fault

2 → [2 0 1] -> Fault

0 → [2 0 1] -> HIT

3 → [2 3 1] -> Fault

0 → [2 3 0] -> Fault

4 → [4 3 0] -> Fault

2 → [4 2 0] -> Fault

3 → [4 2 3] -> Fault

0 → [0 2 3] -> Fault

3 → [0 2 3] -> HIT

Total faults = 10

--- LRU ---

7 $\rightarrow$ [7 -1 -1] Fault0 $\rightarrow$ [7 0 -1] Fault1 $\rightarrow$ [7 0 1] Fault2 $\rightarrow$ [2 0 1] Fault0 $\rightarrow$ [2 0 1] HIT3 $\rightarrow$ [2 0 3] Fault0 $\rightarrow$ [2 0 3] HIT4 $\rightarrow$ [4 0 3] Fault2 $\rightarrow$ [4 0 2] Fault3 $\rightarrow$ [4 3 2] Fault0 $\rightarrow$ [0 3 2] Fault3 $\rightarrow$ [0 3 2] HIT

total faults = 9

----- Optional -----

7 $\rightarrow$ [7 -1 -1] Fault0 $\rightarrow$ [7 0 -1] Fault1 $\rightarrow$ [7 0 1] Fault2 $\rightarrow$ [2 0 1] Fault0 $\rightarrow$ [2 0 1] Fault3 $\rightarrow$ [2 0 2] HIT0 $\rightarrow$ [2 0 3] ~~Fault~~ HIT4 $\rightarrow$ [2 4 3] ~~HIT~~ Fault2 $\rightarrow$ [2, 4, 3] ~~Fault~~ HIT3 $\rightarrow$ [2 4 3] ~~HIT~~ ~~Fault~~ HIT0 $\rightarrow$ [0 4 3] ~~H~~ ~~HIT~~ Fault3 $\rightarrow$ [0 4 3] HIT

total faults = 7

==

26/5/26